



ULM UNIVERSITY OF APPLIED SCIENCES
FACULTY OF COMPUTER SCIENCE

Thesis Title

A thesis submitted in partial fulfillment of the requirements for the degree of
Master of Science in Computer Science

by

Mohamad Arabi

Computer Science

Faculty of Computer Science, Ulm University of Applied Sciences, 2026

April

2026

Supervised by

Prof. Dr. Traub

Prof. Dr. [Co-supervisor]

Ulm, 2026

Statement

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Use of AI

AI tools were used to polish the text and improve the tone of this thesis. The ideas, design, implementation, and evaluation are my own.

Mohamad Arabi

Signature

.....

Date: 15 April, 2026

Abstract

Security operations teams are flooded with alerts, and most SIEM platforms still leave triage and response to human analysts. This thesis presents a generic SIEM reaction plugin that adds a reasoning layer on top of normal SIEM output. Alerts are normalized, enriched, scored with an explainable risk model, and passed to a planner that produces structured response plans. Each plan is checked by policy rules before any action is executed, and every step is recorded in an audit log.

The system is implemented as a single runnable project with a reasoning-focused interface that shows the full chain: raw alert, normalized alert, score and evidence, plan, policy decision, and execution result. The evaluation uses synthetic and public security datasets and shows strong planning accuracy with consistent, explainable outputs. The result is a practical foundation for moving from rule-heavy SIEM workflows to reasoning-driven response that is auditable, safe, and easy to extend.

Summary

Security operations centres face a structural gap between detection and response. SIEM platforms collect, correlate, and surface alerts—but what happens next remains a manual decision. Commercial SOAR tools execute pre-scripted playbooks, which break down on novel attack patterns and cannot reason about threat severity or asset criticality.

This thesis fills that gap with an intelligent reaction layer that sits downstream of a SIEM and upstream of any action. The system takes a normalised alert, enriches it with asset, identity, and threat-intelligence context, produces an explainable threat assessment, generates a structured response plan, and passes every proposed action through a deterministic policy engine before execution. All decisions are recorded in a structured audit log linking each action to its evidence.

The architecture is a modular monolith: a C# core engine enforces the orchestration pipeline and safety constraints, while a Python intelligence service provides scoring and planning via HTTP. The threat scorer combines TF-IDF logistic regression with contextual enrichment boosts. The planner is a custom 270K-parameter multi-head neural network that classifies strategy, predicts action sets, and regresses priority—resolving entity parameters deterministically rather than generating them, eliminating the hallucination risk that made LLM-based planning unsafe for live execution.

The planner achieves 96.4% strategy accuracy and 97.2% action F1 in end-to-end pipeline evaluation—matching a 250-million-parameter fine-tuned seq2seq model at under 5 milliseconds inference time. The policy engine guarantees that no model error can cause a hard-denied action to execute. The full system runs on a single development machine and exposes every stage of the decision pipeline for analyst review.

Acknowledgment

I would like to express my deepest appreciation to all those who contributed to the successful completion of my master's thesis. Foremost, I extend my heartfelt appreciation to my esteemed advisor, Prof. Dr.-Ing. Traub. His guidance, insights, counsel, and mentorship were instrumental in shaping this research.

I also thank Prof. [Second Examiner], who graciously served as the second examiner. Lastly, I would like to thank all individuals and resources, too numerous to name, who contributed indirectly through their research, books, articles, and online sources.

Mohamad Arabi
Department of Computer Science
Technische Hochschule Ulm
Ulm, Germany
15 April, 2026

Table of Contents

Abstract	iii
Summary	iv
Acknowledgment	v
Table of Contents	vi
List of Figures	x
List of Algorithms	xi
List of Abbreviations	xii
List of Symbols	xiii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Thesis Structure	2
2 Background and Related Work	4
2.1 Cybersecurity Monitoring and Security Operations Centers	4
2.2 Security Information and Event Management (SIEM)	4
2.2.1 Log Collection and Normalization	4
2.2.2 Event Correlation	5
2.2.3 Alerting and Incident Response	5
2.3 Limitations of Current SIEM Systems	5
2.3.1 Rule-Based Detection	5
2.3.2 Alert Overload and False Positives	6
2.3.3 Scalability and Data Volume	6
2.3.4 Dependence on Human Analysts	6
2.4 Machine Learning Approaches to Intrusion Detection	6
3 System Requirements	8
3.1 Functional Requirements	8
3.2 Non-Functional Requirements	8
3.3 Security and Safety Requirements	9

4	System Design and Architecture	10
4.1	Design Goals	10
4.2	Key Architectural Decisions	10
4.2.1	Modular Monolith over Multi-Agent Architecture	10
4.2.2	Two-Process Boundary: C# Orchestration and Python Intelligence	11
4.2.3	Interface-Driven Design	11
4.2.4	Policy as a Local, Synchronous Gate	11
4.2.5	Structured Prediction over Free-Form Planning	11
4.3	System Overview	11
4.4	Event Ingestion Layer	12
4.5	Normalization and Enrichment	12
4.6	Threat Scoring Component	12
4.7	Decision Planning Component	13
4.8	Policy and Safety Layer	13
4.9	Action Execution and Response	13
4.10	Audit and Explainability Components	14
4.11	Cross-Cutting Concerns	14
5	Implementation	15
5.1	Technology Stack	15
5.1.1	Orchestration Layer: .NET 9 / C#	15
5.1.2	Intelligence Layer: Python / FastAPI	15
5.1.3	Model Training	15
5.1.4	Persistence and Configuration	16
5.2	Core Engine Implementation	16
5.2.1	Key Interfaces	16
5.2.2	Normalization and Enrichment	17
5.2.3	Webhook and Polling Modes	17
5.3	Intelligence Service	17
5.4	Model Registry and Profile-Based Backend Selection	18
5.5	Threat Scoring Component	18
5.6	Decision Planning Component	18
5.7	Policy and Safety Layer	19
5.8	Execution Pipeline	19
5.9	Auditing and Case Management	19
5.10	Deployment and System Setup	19
6	Dataset Pipeline and Training	20
6.1	Security Datasets and Their Roles	20
6.1.1	MITRE ATT&CK (STIX)	20
6.1.2	Mordor / Security Datasets	20
6.1.3	CIC-IDS2018	21
6.1.4	DARPA Transparent Computing (TC)	21
6.1.5	Synthetic Scenario Generation	21
6.2	Dataset Processing and Normalisation	21
6.2.1	Schema Mapping	21
6.2.2	Label Assignment and Severity Calibration	21

6.2.3	Action Balancing	22
6.3	Training Data Format Evolution	22
6.3.1	Version 1: Full JSON Prompt	22
6.3.2	Version 2: Canonical Flat Format	22
6.3.3	Version 3: Compact Pipe-Delimited Format (Final)	22
6.4	The Scorer: From Non-Functional Baseline to Calibrated Classifier	22
6.4.1	Attempt 1: Zero-Shot LLM (baron-security)	22
6.4.2	Attempt 2: QLoRA Fine-Tuning on Foundation-Sec-8B	23
6.4.3	Attempt 3: TF-IDF Classifier Pipeline (Final)	23
6.5	The Planner: Four Approaches to Structured Prediction	23
6.5.1	Attempt 1: Zero-Shot LLM	23
6.5.2	Attempt 2: Foundation-Sec-8B QLoRA Fine-Tuning	23
6.5.3	Attempt 3: Seq2Seq Fine-Tuning (flan-t5)	23
6.5.4	Attempt 4: Custom Multi-Head MLP (Final)	23
6.6	Evaluation and Results Comparison	24
6.7	Integration with the Intelligence Service	24
7	Evaluation	25
7.1	Evaluation Setup	25
7.1.1	Dataset and Split	25
7.1.2	Evaluation Metrics	25
7.1.3	Evaluation Environment	26
7.2	Detection Performance	26
7.2.1	Scorer Results	26
7.2.2	Calibrated Confidence Behavior	26
7.2.3	Comparison with LLM Baselines	26
7.3	Response Planning Evaluation	27
7.3.1	Summary of Model Progression	27
7.3.2	Interpretation of Remaining Errors	27
7.3.3	Policy Engine Correctness	27
7.4	Example Incident Scenarios	27
7.4.1	Scenario 1: Malware Hash Detection — Autonomous Containment	27
7.4.2	Scenario 2: Brute Force Attempt — Conditional Containment	28
7.4.3	Scenario 3: Suspicious Process — Investigative Hold	28
7.4.4	Scenario 4: Benign Noise — Autonomous Dismissal	28
7.5	Discussion of Results	29
8	Discussion	30
8.1	Advantages of the Proposed System	30
8.1.1	Explainability as a First-Class Output	30
8.1.2	Safety Independent of Model Quality	30
8.1.3	Operational Feasibility	30
8.1.4	Model-Agnostic Architecture	31
8.2	Comparison with Traditional SIEM and SOAR Approaches	31
8.2.1	Against SIEM Platforms	31
8.2.2	Against Playbook-Based SOAR	31
8.2.3	Against Generative LLM-Based Planning	31

8.3	Limitations	31
8.3.1	Evaluation on Synthetic Data Only	31
8.3.2	Limited Alert Type Coverage	32
8.3.3	Simulated SIEM Connector	32
8.3.4	Manually Authored Policy Rules	32
8.3.5	Feedback Loop Not Fully Closed	32
8.4	Threats to Validity	32
8.4.1	Internal Validity	32
8.4.2	External Validity	32
8.4.3	Construct Validity	33
8.4.4	Statistical Validity	33
9	Conclusion and Future Work	34
9.1	Summary of Contributions	34
9.2	Future Research Directions	34
	References	36
A	Appendix	39
A.1	Additional Figures	39
A.2	Configuration Details	39

List of Figures

List of Algorithms

List of Abbreviations

SIEM	S ecurity I nformation and E vent M anagement
SOAR	S ecurity O rchestration, A utomation, and R esponse
SOC	S ecurity O perations C enter
EDR	E ndpoint D etection and R esponse
XDR	eX tended D etection and R esponse
IOC	I ndicator o f C ompromise
TTP	T actics, T echniques, and P rocedures
ATT&CK	A dversarial T actics, T echniques, and C ommon K nowledge
LLM	L arge L anguage M odel
ML	M achine L earning
JSON	J ava S cript O bject N otation
API	A pplication P rogramming I nterface
STIX	S tructured T hreat I nformation eX pression
TAXII	T ruste d A utomated eX change of I ndicator I nformation

List of Symbols

R	risk score (overall)
b	bias term in risk model
w_i	feature weight i
x_i	feature value i
m	context multiplier
c	confidence (0–1)
S	severity (0–100)
P	response priority (0–100)

Introduction

1.1 Motivation

Security Operations Centers (SOCs) rely on SIEM platforms to collect and centralize security logs from many different systems. [1] Theoretically, this should allow analysts to have a complete view of what is going on with the infrastructure. In practice, the SOC analysts deal with a constant flow of alerts. Many of them are noisy, duplicated, and of poor quality. Moreover, they still need to be examined and the decision made which ones actually signal a threat.[2]

This situation creates several problems. Alert queues accumulate quickly, response time increases, and the analyst works under the pressure of the possibility that an important alert will be missed. This leads to a phenomenon referred to as alert fatigue, where the information overload is more challenging to deal with than the actual issues themselves.[3]

To understand this, let's consider an example. For instance, if there are several unsuccessful login attempts, it may be indicative of a brute-force attack. However, it may also be indicative of a misconfigured script. A human analyst will be able to understand the difference between the two, but only if the necessary context is provided and it is presented in an understandable manner. So, it's not just the matter of detecting, but it's also about reasoning, prioritization, and determining the course of action.

1.2 Problem Statement

Most SIEM environments rely on rule-based detection. As rule sets gets bigger over time, they tend to overlap and produce redundant alerts. This increases the workload and makes it harder to identify the alerts that actually require attention. [4] Even in

environments that introduce automation, studies show that SOC workflows still depend a lot on human judgment when deciding how to respond to incidents. [3]

This creates a gap in current SIEM workflows. While SIEM platforms are effective at collecting and detecting events, they provide limited support for reasoning about response actions. While alerts are generated, the system generally does not create a plan for what should be done next. Additionally, there is no capturing of analyst feedback to improve future decisions.

This thesis attempts to fill this gap by proposing a generic reaction layer. This layer will be used after the alert normalization process. It will evaluate the alerts, create explainable risk scores, and create a plan for the response. The policy will be used before the actions are taken, and an audit trail will be used for the decisions.

1.3 Research Objectives

The work presented in this thesis is guided by the following objectives:

- Design a layered architecture that separates orchestration from intelligence and allows detection models to be replaced without disrupting the surrounding SIEM pipeline.
- Implement an explainable risk scoring engine that uses evidence-based features along with contextual information and offers an explanation for each score.
- Create a structured planning module that offers response plans, policy conformance for those plans, and rollbacks.
- Integrate analyst feedback and investigation traces so that the system can learn from past investigations and suggest useful next steps.
- Demonstrate the approach through an end-to-end prototype that includes a runnable system and an interface focused on reasoning and investigation support.

1.4 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews background concepts and related work in SIEM systems, alert correlation, and response automation. Chapter 3 defines the system requirements. Chapter 4 explains the system design and architecture. Chapter 5 explains the implementation details of the system. Chapter 6

explains the dataset pipeline and the training approach used for the models. Chapter 7 explains the evaluation of the system. It includes the results of the system. Chapter 8 explains the implications of the system. Finally, Chapter 9 concludes the thesis. It explains the possible directions for future work.

Background and Related Work

2.1 Cybersecurity Monitoring and Security Operations Centers

Modern organisations generate massive volumes of digital events. These include network flows, logins, process executions, and file system changes. A Security Operations Center (SOC) is responsible for monitoring this activity, triaging suspicious events, and coordinating response.

The classic incident response lifecycle is documented by NIST in SP 800-61. [5] It separates work into preparation, detection and analysis, containment and recovery, and post-incident review. The detection and analysis phase is usually the most expensive because it depends on human interpretation and on the quality of monitoring infrastructure.

SOC teams operate under high pressure. Tilbury and Flowerday report sustained alert overload and a tension between automation and human oversight. [3] Their conclusion is practical: automation should reduce mechanical workload, not remove human judgment from high-impact decisions. This thesis follows that principle.

2.2 Security Information and Event Management (SIEM)

2.2.1 Log Collection and Normalization

A SIEM platform collects logs from heterogeneous sources and makes them queryable for analysts and correlation engines. NIST SP 800-92 describes SIEM log management as a pipeline: collect, parse, normalize, and store. [1]

Normalization is a core requirement. Windows Event Logs, syslog, CEF, and LEEF all use different field layouts. A field called **severity** in one source can mean an alert

priority, while in another it reflects a risk assessment. Normalization mistakes propagate into every downstream stage. For that reason, the system in this thesis treats normalization as a separate, explicit pipeline stage that can be updated without touching the intelligence layer. This aligns with the modularity principle in [1].

2.2.2 Event Correlation

Single events are rarely enough. A single failed login is usually noise, but many failed logins in a short time can indicate brute force. Correlation connects events across time and sources to create a higher-level alert.

Valeur et al. [6] describe correlation as a structured process: preprocessing, duplicate removal, alert fusion, correlation, false-positive reduction, and session reconstruction. Their work highlights a practical issue: what practitioners call “correlation” is often several different operations mixed together. Separating these stages makes the system easier to reason about and improves control over false positives.

2.2.3 Alerting and Incident Response

The output of correlation is an alert. An analyst must decide whether it is real, how severe it is, and what response is appropriate. The process is formalized in [5], but in practice prioritization varies by analyst and by workload. [3] This inconsistency motivates machine-assisted scoring and structured response planning.

2.3 Limitations of Current SIEM Systems

2.3.1 Rule-Based Detection

Rule-based correlation does not generalize. Rules encode what an expert anticipated, but they fail on novel or subtle patterns. Attackers exploit this with “living-off-the-land” techniques that use legitimate tools (PowerShell, WMI, certutil) to avoid signatures. [7]

Rule sets also accumulate and overlap. Sarhan et al. show that redundant rules generate unnecessary alerts and that rule hygiene has a measurable effect on alert quality. [4] This is not only a volume problem; it is a correctness problem.

2.3.2 Alert Overload and False Positives

High false-positive rates lead directly to alert fatigue. Ban et al. report deployments where false positives dominate the alert stream, forcing analysts to ignore alerts just to keep up. [2]

Axelsson’s base-rate analysis explains why this is structural. [8] Even a detector with 99% sensitivity and 99% specificity can produce more false alerts than true alerts when the base rate of attacks is low. This makes precision a central requirement for any operational system.

2.3.3 Scalability and Data Volume

Enterprise log volume can reach terabytes per day. [1] Scaling ingestion is possible, but it does not reduce analyst workload. Alert volume depends on rule behavior and correlation quality. [4] Without better reasoning, more data simply means more alerts.

2.3.4 Dependence on Human Analysts

SOC effectiveness is still bounded by human availability. Tilbury and Flowerday describe analyst shortage, turnover, and burnout as persistent constraints. [3] This reinforces the need for systems that handle mechanical triage while preserving human control of high-impact decisions.

2.4 Machine Learning Approaches to Intrusion Detection

Rule limitations have driven research into machine learning for intrusion detection and alert analysis. Two main paradigms dominate the literature.

Anomaly-based detection learns a model of normal behavior and flags deviations. Garcia-Teodoro et al. survey anomaly detection methods and show that while these systems can detect novel attacks, they often generate high false-positive rates because “abnormal” includes many benign but unusual behaviors. [9]

Signature-based ML treats detection as a classification problem trained on labeled attacks. This can produce high precision for known patterns but struggles on new attack variants, similar to rules.

Sommer and Paxson provide a critical analysis of ML for network intrusion detection. [10] They argue that distribution shift, attacker adaptation, and data quality issues limit the operational reliability of ML systems. Their critique motivates the design of systems that emphasize explainability, policy constraints, and bounded automation rather than blind reliance on model accuracy.

System Requirements

3.1 Functional Requirements

The system must ingest alerts from external SIEM sources, normalize them to a common schema, enrich them with context, and produce ranked incident candidates. A scoring component must generate an explainable risk score and confidence for each incident, together with reason codes and context multipliers. A planning component must produce a structured response plan with a strategy, priority, actions, and rollback steps. Each plan must be validated against explicit policy rules before execution, and the execution stage must record action outcomes and errors.

The system must support analyst feedback. An analyst can mark an incident as true positive, false positive, or needs more information, and optionally override severity. Feedback events must be recorded in an append-only log and used to update model weights in a bounded, auditable way. The system must also track investigation traces and use them to recommend next steps. A user interface must expose raw alerts, normalized alerts, scoring explanations, plans, policy checks, execution status, and audit logs.

3.2 Non-Functional Requirements

The architecture must be modular to allow the intelligence models to be swapped without modifying the orchestration code. The interface between layers must be explicit. The system must be able to be executed as a single project to facilitate demonstrations or repeated experiments. The explanations must be human-readable. The explanations must be consistent to compare inputs. The implementation must be maintainable. There must be a clear separation between the domain logic, the application services, the infrastructure, and the UI. There must be structured logging to facilitate audits.

3.3 Security and Safety Requirements

Automation must be bounded. The actions proposed by the planner should be subject to policy rules such as allow/deny lists, confidence thresholds, and asset criticality limits. High-impact actions should be subject to approval gates/dual control, and each automated action should have a rollback action.

The execution layer must validate parameters, enforce entity presence checks, and refuse execution when required context is missing or policy checks fail. Rate limits and idempotency safeguards must prevent repeated execution under noisy alerts. If confidence is too low, or policy approval is not met, the system must fall back to recommendation-only behavior.

Model risk must be controlled. Feedback-driven weight updates must be bounded and logged in an append-only audit trail. Model output should be traceable after updates. Support should be provided for drift detection and periodic offline recalibration. Data handling should conform to the principle of least privilege for connectors and executors, and sensitive field minimization or redaction should be employed where possible. All policy evaluations and execution outcomes should be logged with timestamps and actor identity.

System Design and Architecture

4.1 Design Goals

The architecture was built around a small set of goals that reflect how SOC systems are used in practice.

Explainability first. Every score, strategy, and action must have a reason that an analyst can read. This rules out black-box outputs and forces the system to carry evidence and rationale at every stage.

Safety before speed. A wrong containment action can be worse than a missed alert. The design therefore enforces policy checks and approval gates before execution, even if that adds latency.

Modularity and replaceability. Models and connectors change quickly. Each component must be swappable without rewriting the rest of the pipeline.

Runs on one machine. The system must be runnable end-to-end on a single developer machine for demos and testing. External services are optional, not required.

Reproducibility. Given the same input and configuration, the system should produce the same output. This requires deterministic post-processing and a clear audit trail.

4.2 Key Architectural Decisions

4.2.1 Modular Monolith over Multi-Agent Architecture

A multi-agent design was considered. It would split ingestion, scoring, planning, and execution into separate autonomous agents. While this can scale, it introduces distributed

failure modes and makes reasoning harder to audit. In security response, accountability matters more than autonomy. For that reason, the core engine is a modular monolith with a single, synchronous pipeline. Every decision flows through one call chain that can be logged end-to-end.

4.2.2 Two-Process Boundary: C# Orchestration and Python Intelligence

The system is intentionally split into two processes. The orchestrator runs in C# (.NET) because it handles state, policy enforcement, and long-running reliability. The intelligence service runs in Python because the ML ecosystem lives there. The two communicate over HTTP with a strict JSON contract. This boundary keeps each side simple: the orchestrator stays stable, and the intelligence layer can evolve quickly.

4.2.3 Interface-Driven Design

Each stage is defined by an interface: connector, normalization pipeline, scorer, planner, policy engine, execution pipeline, audit logger, and case manager. This makes the system testable with stubs and components replaceable without modifying the orchestrator.

4.2.4 Policy as a Local, Synchronous Gate

Policy evaluation is inside the core engine, not a remote call. This guarantees that no action can bypass safety checks due to network failures or model errors. Hard denials are applied before any approval workflow, which prevents unsafe actions from entering the approval queue.

4.2.5 Structured Prediction over Free-Form Planning

Early experiments with free-form JSON planning showed two problems: invalid output and hallucinated entities. The system therefore treats planning as structured prediction. The model selects a strategy and action types, while entity parameters are filled deterministically from the normalized alert. This removes hallucination by design.

4.3 System Overview

The system processes alerts in a linear pipeline:

Ingest -> Normalize -> Enrich -> Score -> Plan -> Policy -> Execute -> Audit

Each stage emits structured data and is logged. The pipeline is synchronous and deterministic, which makes it easier to explain and to debug.

4.4 Event Ingestion Layer

The ingestion layer pulls alerts from external SIEMs or accepts them via webhook. Two connectors are implemented: a Wazuh connector for real SIEM input and a simulator for controlled demos. Each connector outputs a **RawAlert** object with the original payload and metadata such as source, alert ID, and timestamp.

4.5 Normalization and Enrichment

Normalization transforms non-uniform alert data formats to a standardized format. Consider the Wazuh alert format that stores the IP address of the source in `data.srcip`, and the Sentinel alert format that stores the IP address of the source as a list of entities. The mapping registry normalizes these differences. Enrichment provides contextual information like criticality of assets, threat intel hits, and identity privileges. These fields are optional; if an enrichment source is missing, the system continues with null values rather than failing the entire pipeline.

4.6 Threat Scoring Component

The scoring stage produces a **ThreatAssessment** with severity, confidence, hypothesis, and evidence. The default scorer is a TF-IDF based classifier with calibrated confidence thresholds. A hybrid mode can add an LLM explanation when confidence is borderline. A rule-based fallback ensures that the pipeline always returns a score.

The scorer is informational, not authoritative. Its output drives planning and policy, but policy rules can still block actions even for high scores.

4.7 Decision Planning Component

Planning converts an assessment into a structured **DecisionPlan**. Each plan includes a strategy, priority, a list of actions, and rationale text. The system uses five strategy labels:

- **Ignore**: The alert is assessed as benign noise; no action is taken.
- **NotifyOnly**: Low severity or confidence; notify the SOC and open a ticket.
- **Investigate**: Warrants investigation but no automated containment.
- **ContainAndCollect**: Active threat confirmed; containment plus forensics.
- **Contain**: Immediate high-confidence threat; containment is the priority.

The planner is model-driven, but its output is sanitized to enforce schema validity, safe actions at low confidence, and parameter completeness. This keeps the plan consistent even when models fail or return partial output.

4.8 Policy and Safety Layer

The policy engine analyzes each proposed action and returns a decision of Approved, Pending, or Denied. Hard constraints are checked first (missing entities, forbidden actions, low confidence). Approval gates are applied next, using risk, environment, and asset criticality factors. All policy decisions are logged with explanations so that analysts can trace exactly why a particular action was allowed or denied.

4.9 Action Execution and Response

Approved actions are routed to executors (ticketing, notification, firewall, host isolation). Each executor validates parameters and returns a structured result. In demo mode, executors simulate the action and only write audit entries. This keeps the system safe while still showing a complete execution pipeline.

4.10 Audit and Explainability Components

Every stage writes structured audit events to JSONL. These logs include the raw alert, normalized alert, assessment, plan, policy decision, and execution results. The same data powers the reasoning console UI, which allows the full decision trail to be viewed and replayed.

4.11 Cross-Cutting Concerns

Configuration. All services are configured via environment variables and a single `.env` file, making model switching and demo setup fast.

Resilience. Each stage is isolated; a failure in one alert does not crash the pipeline for the entire batch.

Extensibility. New connectors, actions, or models can be added by implementing a single interface and registering it with the container or model registry.

Implementation

This chapter explains how the system is built and how the main components interact. The focus is on practical engineering choices and the reasons behind them.

5.1 Technology Stack

5.1.1 Orchestration Layer: .NET 9 / C#

The orchestration engine is written in C# and targets `net9.0`. This layer runs the end-to-end pipeline, applies policy rules, and records audit logs. .NET is a good fit because it provides strong typing, stable async I/O, and a built-in dependency injection container. These features make the pipeline easy to test and safe to extend.

The engine is a console application, not a web server. It can poll a SIEM API or listen to a webhook, but its main job is alert processing, not serving HTTP traffic.

5.1.2 Intelligence Layer: Python / FastAPI

The intelligence service is a Python FastAPI app. Python is used because all ML tooling (PyTorch, scikit-learn, Hugging Face) is built around it. FastAPI provides clear request validation and OpenAPI docs, which helps keep the interface between the two services stable and visible.

5.1.3 Model Training

The planner is trained with PyTorch. The scorer uses scikit-learn with TF-IDF features. Both produce small model artifacts (`.pt` for PyTorch and `.joblib` for scikit-learn) that can be loaded directly at runtime without a separate model server.

5.1.4 Persistence and Configuration

Audit logs are stored as JSONL for simplicity and easy inspection. Cases are stored in SQLite to keep the system self-contained. Configuration is environment-based via a `.env` file, so switching models or connectors does not require recompiling the engine.

5.2 Core Engine Implementation

The orchestrator runs a strict, linear pipeline. Each stage is isolated behind an interface so it can be swapped or mocked in tests. The central loop is simple and explicit:

```
RawAlert[] alerts = await connector.PullAlertsAsync(request, ct);

foreach (RawAlert raw in alerts)
{
    EnrichedAlert enriched = await normalization.ProcessAsync(raw, ct);
    ThreatAssessment assessment = scorer.Score(enriched);
    DecisionPlan plan = planner.Plan(enriched, assessment, context);
    PolicyDecision decision = policy.Evaluate(plan, assessment, enriched, context);
    ExecutionReport report = await execution.ExecuteAsync(
        enriched, assessment, plan, decision, execContext, ct);

    audit.Log(raw, enriched, assessment, plan, decision, report);
    caseManager.OpenOrUpdate(assessment, plan, decision);
}
```

Failures are contained at the stage boundary. For example, if normalization fails, the system logs the error and moves to the next alert. Scoring failures fall back to a low-confidence rule-based score rather than stopping the pipeline. This is critical in real SOC workflows where partial failures are normal.

5.2.1 Key Interfaces

The pipeline is built around a small set of interfaces:

- **ISiemConnector**: pulls alerts or accepts webhook input.

- **INormalizationPipeline**: maps raw alerts to a canonical schema and enriches them.
- **IThreatScorer**: returns `ThreatAssessment` objects.
- **IPlanner**: returns `DecisionPlan` objects.
- **IPolicyEngine**: evaluates plans and returns per-action decisions.
- **IExecutionPipeline**: executes approved actions and returns an `ExecutionReport`.
- **IAuditLogger** and **ICaseManager**: persist the decision trail.

This design makes the system easy to test and easier to extend. A new connector or planner can be introduced without changing the orchestrator.

5.2.2 Normalization and Enrichment

Normalization translates alert fields from different SIEMs into a common schema. For example, a Wazuh alert stores the source IP under `data.srcip`, while a Sentinel alert uses a nested `entities` list. A mapping registry handles these differences so downstream components always see `srcIp`, `hostId`, `username`, and similar fields in the same place.

Enrichment adds context from JSON-based inventories (assets, identities, threat intel). These providers are optional. If a lookup fails, the pipeline keeps going with null values rather than aborting.

5.2.3 Webhook and Polling Modes

The engine can run in polling mode (periodic pulls from the SIEM API) or webhook mode (alerts pushed to a local listener). For demos, webhook mode is used with a replay script that posts recorded alerts at a controlled pace.

5.3 Intelligence Service

The intelligence service exposes two endpoints: `POST /v1/score` and `POST /v1/plan`. It loads models at startup to avoid first-request delays. If a model fails to load, the service falls back to a rule-based scorer or planner instead of crashing.

A small LRU cache is used for scoring responses to reduce redundant inference during demos.

5.4 Model Registry and Profile-Based Backend Selection

Models are selected through a JSON registry file (`intelligence/models/registry.json`). Each profile defines a scorer and planner backend. Profiles can be switched by changing one config value or by passing a header per request. This makes A/B evaluation simple and avoids code changes when models are swapped.

5.5 Threat Scoring Component

The scorer outputs a **ThreatAssessment** with severity, confidence, hypothesis, and evidence. The default scorer is a TF-IDF based classifier. It predicts one of five severity labels and maps that label to a numeric score.

A calibrated confidence floor is applied when the top class is clearly dominant. For example, if the classifier is decisive on a “critical” label, the reported confidence is raised to a minimum floor rather than exposing a low raw probability. This helps policy thresholds behave sensibly.

Enrichment can also raise the score. A known-bad TI reputation or a critical asset will bump confidence and severity using simple max-based rules. This makes the system behave more like an analyst: strong external evidence should override a cautious model.

Hypothesis and evidence strings are generated deterministically from alert fields and enrichment. This avoids LLM hallucination and keeps explanations reproducible.

5.6 Decision Planning Component

Planning is treated as structured prediction. The model selects a strategy, action types, and a priority. Action parameters are filled from the normalized alert, not generated by the model. This prevents hallucinated IPs or hostnames.

The primary planner is a custom multi-head MLP trained on 8,644 structured examples. It predicts:

- Strategy (5-class softmax)
- Action set (multi-label sigmoid over 8 actions)
- Priority (regression scaled to 0–100)

The feature vector combines structured fields (alert type, severity, confidence, entity presence) with keyword flags and TF-IDF features from the hypothesis and evidence text. This combination proved more reliable than a large seq2seq model and runs in a few milliseconds on CPU.

A sanitization pass enforces schema validity, adds safe actions if needed, and strips actions that are infeasible due to missing entities. This ensures stable output even when model predictions are imperfect.

5.7 Policy and Safety Layer

Policy evaluation is local and synchronous. Each planned action is checked against hard constraints (missing entities, forbidden actions, low confidence) and then evaluated for approval thresholds. The result is **Approved**, **Pending**, or **Denied**. This guarantees that model output never bypasses safety rules.

5.8 Execution Pipeline

The execution pipeline routes approved actions to executors (ticketing, notification, firewall, host isolation). Each executor validates parameters and returns a structured result. In demo mode, executors simulate actions and only write audit entries.

5.9 Auditing and Case Management

Every stage writes an audit record to JSONL. The case manager stores the current state of each incident in SQLite. This data powers the reasoning console and allows full replay of the decision chain.

5.10 Deployment and System Setup

The system starts from a single command once `.env` is configured. The C# engine can launch the Python intelligence service as a child process, wait for its health check, and then start processing alerts. This keeps the demo setup simple and repeatable.

Dataset Pipeline and Training

This chapter explains how the training data was built and how the scorer and planner models evolved. The process was not linear. Several approaches failed before the final pipeline stabilized.

6.1 Security Datasets and Their Roles

No single dataset covers all alert types, entity patterns, and response scenarios. The pipeline therefore combines multiple sources plus a synthetic generator.

6.1.1 MITRE ATT&CK (STIX)

The MITRE ATT&CK framework provides a structured taxonomy of tactics and techniques. [7] It is used to ground labels and to generate realistic hypothesis text. The STIX data is downloaded and parsed so that technique IDs and descriptions can be attached to alerts.

6.1.2 Mordor / Security Datasets

Mordor provides real telemetry from attack emulations. [11] It contributes realistic entity values (process names, registry paths, hostnames) and realistic co-occurrence patterns. For example, a Mimikatz run often appears with both process creation and network events.

6.1.3 CIC-IDS2018

CIC-IDS2018 is a large labeled network-flow dataset. [12] It provides port scan and brute force patterns at scale. It is especially useful for calibrating severity ranges for network-based alerts.

6.1.4 DARPA Transparent Computing (TC)

DARPA TC provides provenance-rich traces across multiple attack stages. [13] These traces help generate alerts that include host, process, and file hash entities together, which are important for containment actions such as `QuarantineFile`.

6.1.5 Synthetic Scenario Generation

Real datasets are imbalanced. Some alert types appear rarely, but the system must still handle them. A synthetic generator fills the gaps. It produces alerts with entity-conditional actions so the planner learns when actions are feasible. For example, `BlockIp` is only labeled when a source IP exists. This avoids teaching the model impossible actions.

6.2 Dataset Processing and Normalisation

All sources are mapped into the same canonical schema used by the production pipeline. This keeps training inputs aligned with inference inputs.

6.2.1 Schema Mapping

Each data source is mapped into a compact JSON structure with `type`, `severity`, `sourceSiem`, `entities`, and `rawPayload`. Missing fields are explicitly set to `null` rather than omitted. This helps the planner learn the difference between “missing” and “not applicable.”

6.2.2 Label Assignment and Severity Calibration

Severity ranges are defined per alert type and then sampled within those ranges. For example, malware hash alerts receive higher severity than port scans. These ranges were calibrated using distributions from CIC-IDS2018 and Mordor.

6.2.3 Action Balancing

Early datasets were action-imbalanced: `OpenTicket` and `Notify` dominated, while containment actions were rare. The dataset builder therefore upsamples under-represented actions until each action type appears at a minimum frequency. This greatly improved planner recall for containment actions.

6.3 Training Data Format Evolution

Three formats were tested before settling on a compact prompt.

6.3.1 Version 1: Full JSON Prompt

The first format used full JSON blobs in the prompt and completion. This was too long for seq2seq models and produced unstable outputs.

6.3.2 Version 2: Canonical Flat Format

The second format flattened the alert into key-value lines. This was more stable but still verbose and sensitive to minor formatting changes.

6.3.3 Version 3: Compact Pipe-Delimited Format (Final)

The final format uses a compact, pipe-delimited prompt with a strict output schema. It is short, consistent, and easy to parse. This format is used for both seq2seq and custom NN training.

6.4 The Scorer: From Non-Functional Baseline to Calibrated Classifier

6.4.1 Attempt 1: Zero-Shot LLM (baron-security)

A zero-shot LLM baseline was tested first. It failed to produce stable severity outputs and often returned empty or overly safe responses.

6.4.2 Attempt 2: QLoRA Fine-Tuning on Foundation-Sec-8B

QLoRA fine-tuning was attempted to improve LLM scoring. [14–16] While it produced plausible text, inference was slow and output formats were inconsistent for automated evaluation.

6.4.3 Attempt 3: TF-IDF Classifier Pipeline (Final)

The final scorer is a TF-IDF classifier trained on the canonical prompt text. It is fast, stable, and easy to calibrate. The implementation uses scikit-learn. [17] Confidence floors are applied when predictions are decisive, which aligns the output with policy thresholds.

6.5 The Planner: Four Approaches to Structured Prediction

6.5.1 Attempt 1: Zero-Shot LLM

A zero-shot LLM planner produced invalid JSON and hallucinated entity values. This was unsafe and unreliable.

6.5.2 Attempt 2: Foundation-Sec-8B QLoRA Fine-Tuning

QLoRA fine-tuning produced better plans but still suffered from format instability and slow inference. It was unsuitable for near-real-time use. [14–16]

6.5.3 Attempt 3: Seq2Seq Fine-Tuning (flan-t5)

Seq2seq planning with Flan-T5 produced valid output in many cases. [18] However, inference was slow (hundreds of milliseconds) and still required sanitization.

6.5.4 Attempt 4: Custom Multi-Head MLP (Final)

The final planner is a small multi-head MLP trained with PyTorch. [19] It predicts strategy, action set, and priority from a structured feature vector. Feature engineering (keyword flags and TF-IDF) was the key factor in reaching high accuracy. The model is fast and stable and aligns well with the sanitization rules used in production.

6.6 Evaluation and Results Comparison

The custom MLP achieves the best balance of accuracy and speed. On the 865-example test set, it reaches 96.4% strategy accuracy and 97.2% action F1 in the pipeline evaluation. Seq2seq models reach comparable accuracy but at far higher latency and with larger resource requirements. The final choice favored stability and speed because the system is designed for operational use, not just benchmark performance.

6.7 Integration with the Intelligence Service

All models are exposed through the same FastAPI service. The model registry selects the active profile at runtime, and models are loaded once at startup. This allows rapid switching between the custom MLP, the seq2seq planner, and rule-based baselines without code changes.

Evaluation

This chapter evaluates the threat scorer and the response planner. It also walks through example incidents to show how the full pipeline behaves.

7.1 Evaluation Setup

7.1.1 Dataset and Split

Evaluation uses the synthetic dataset described in Chapter 6. The dataset contains 8,644 labeled examples across five alert types: `MalwareHashOnHost`, `BruteForceUser`, `SuspiciousProcessOnHost`, `PortScanFromIp`, and `BenignNoise`. A fixed random seed (42) is used to split the data into 90% training (7,779 examples) and 10% test (865 examples), with stratification by alert type.

7.1.2 Evaluation Metrics

Scorer evaluation uses two metrics. **Severity class accuracy** measures how often the predicted severity label (benign, low, medium, high, critical) matches the ground truth. **Severity MAE** measures the average absolute error on the 0–100 severity scale.

Planner evaluation uses two metrics. **Strategy accuracy** is the exact-match rate for the strategy label (`ContainAndCollect`, `Contain`, `Investigate`, `NotifyOnly`, or `Dismiss`). **Action F1** is the macro-averaged F1 score over the predicted action set. Rollback actions are stripped from both predictions and targets. Plans are passed through `sanitize_plan` before scoring to reflect real deployment behavior. A **relaxed strategy accuracy** is also reported, where `Contain` and `ContainAndCollect` are treated as near-matches.

7.1.3 Evaluation Environment

Training and evaluation were run on a workstation with an Intel Core i9 CPU, 64 GB RAM, and an NVIDIA RTX 5070 Ti (16 GB VRAM). The intelligence service runs locally. Inference latency is measured at the C# engine, including serialization and HTTP round-trip time.

7.2 Detection Performance

7.2.1 Scorer Results

The TF-IDF classifier achieves **75.0% severity class accuracy** with a **severity MAE of 8.30** on the test set. With five classes, random guessing yields 20% accuracy, so the classifier is far above baseline. The MAE corresponds to less than one severity tier on average.

Inference is sub-millisecond on CPU. The model artifact is 2.3 MB, which makes it easy to load and update.

7.2.2 Calibrated Confidence Behavior

Raw classifier probabilities underestimate decisiveness in multi-class settings. A simple calibration floor is applied: if the top class is clearly dominant, confidence is raised to a minimum floor. This prevents high-severity alerts from being treated as low-confidence by policy rules.

7.2.3 Comparison with LLM Baselines

A zero-shot LLM baseline produced near-zero discrimination and often returned empty outputs. A QLoRA fine-tuned Foundation-Sec model produced plausible severity ordering but was too slow (2–4 seconds per alert) and inconsistent in output format. The TF-IDF classifier provides reliable output at sub-millisecond latency and is therefore used as the default.

7.3 Response Planning Evaluation

7.3.1 Summary of Model Progression

The custom MLP achieves **96.4% strategy accuracy** and **97.2% action F1** on the pipeline evaluation set. The flan-t5-base seq2seq model reaches similar accuracy on a smaller evaluation subset, but at much higher latency. The custom MLP was chosen as the default because it is both fast and reliable.

7.3.2 Interpretation of Remaining Errors

Most errors are minor strategy shifts (Contain versus ContainAndCollect). These are near-misses rather than catastrophic mistakes. The relaxed accuracy (96.8%) captures this effect. Truly severe errors, such as predicting Dismiss for a ContainAndCollect case, are rare.

7.3.3 Policy Engine Correctness

The policy engine is deterministic. It enforces hard constraints regardless of model output, which means unsafe actions are blocked even when a model makes a mistake. This separation is a key safety property of the system.

7.4 Example Incident Scenarios

7.4.1 Scenario 1: Malware Hash Detection — Autonomous Containment

A simulated EDR alert reports a known-bad file hash on a production domain controller.

Type:	MalwareHashOnHost		
Severity:	88	Confidence:	0.91
HostId:	dc01.corp.local		
FileHash:	4a8b9f...e29f		
ProcessName:	svchost_spoofed.exe		
AssetCriticality:	5 (maximum)		

Threat intel confirms the hash and raises the final score to 94. The planner selects `ContainAndCollect` with `IsolateHost`, `KillProcess`, `QuarantineFile`, and `CollectForensics`. Policy approves the actions because entity parameters are present and thresholds are met. The audit log records the full reasoning trail.

7.4.2 Scenario 2: Brute Force Attempt — Conditional Containment

A brute force alert arrives with a source IP but no username.

```
Type:                BruteForceUser
Severity:            72    Confidence: 0.79
SrcIp:               203.0.113.47
Username:            null
AssetCriticality:    3
```

The planner selects `Contain` with `BlockIp` and `OpenTicket`. `DisableUser` is omitted because no username is present. Policy approves `BlockIp` and `OpenTicket`.

7.4.3 Scenario 3: Suspicious Process — Investigative Hold

A suspicious process is detected on an internal workstation.

```
Type:                SuspiciousProcessOnHost
Severity:            61    Confidence: 0.72
HostId:              ws42.corp.local
ProcessName:         certutil.exe
AssetCriticality:    2
```

The planner selects strategy `Investigate` with `CollectForensics` and `OpenTicket`. No containment is executed. The ticket includes the process name and host identity for analyst review.

7.4.4 Scenario 4: Benign Noise — Autonomous Dismissal

A low-confidence internal DNS event is detected.

Type: BenignNoise
Severity: 6 Confidence: 0.94
SrcIp: 10.0.1.10
AssetCriticality: 1

The planner selects **Dismiss** and no actions are executed. The event is logged for audit purposes. This suppresses low-value alerts and reduces analyst load. [2]

7.5 Discussion of Results

The results support four conclusions. First, planning accuracy is driven more by data quality than by model size. Second, structured prediction avoids hallucinated parameters by design. Third, the policy engine provides a hard safety floor regardless of model errors. Fourth, synthetic evaluation is useful for controlled benchmarking, but real-world validation remains a necessary next step. [10]

Discussion

This chapter places the results in context, highlights practical advantages, and outlines limitations and threats to validity.

8.1 Advantages of the Proposed System

8.1.1 Explainability as a First-Class Output

Most systems add explanations after the fact. This system builds explanations into the data model. A **ThreatAssessment** cannot exist without a hypothesis and evidence list. A **DecisionPlan** includes per-action rationale. A **PolicyDecision** records the rule that approved, held, or denied each action. This makes the audit trail a direct record of what the system actually used at decision time, not a reconstruction. [5]

8.1.2 Safety Independent of Model Quality

The policy engine enforces hard constraints even when the model is wrong. Actions that lack required entities or violate thresholds are denied before they can reach execution. This is stronger than typical playbook systems where safety depends on the correctness of the playbook. [3]

8.1.3 Operational Feasibility

The custom MLP planner runs in under 5 ms. The TF-IDF scorer is sub-millisecond. This keeps the total intelligence overhead under 10 ms per alert, which is practical for real-time triage. The seq2seq alternative is far slower, which makes it difficult to deploy at scale. [2]

8.1.4 Model-Agnostic Architecture

The model registry separates model choice from orchestration code. Profiles can be switched without redeploying the engine. This makes experimentation and rollback safe and fast.

8.2 Comparison with Traditional SIEM and SOAR Approaches

8.2.1 Against SIEM Platforms

SIEM platforms collect and correlate logs, but they do not reason about response. [1] They surface alerts and rely on analysts to decide what to do next. This system starts where SIEM stops. It takes normalized alerts and adds scoring, planning, policy checks, and audited execution.

8.2.2 Against Playbook-Based SOAR

SOAR platforms execute predefined workflows. That works well for known cases, but it struggles with new or mixed patterns. [3] The planner in this system learns from data and generalizes across scenarios. It still makes mistakes, but it does not fail silently when no playbook matches.

8.2.3 Against Generative LLM-Based Planning

LLM planning can generate plausible output but often hallucinates entity values. In a response system, a wrong IP or host is a serious safety problem. The structured prediction approach avoids this by design: the model selects action types, and parameters are pulled from the alert context. This keeps actions grounded in real entities.

8.3 Limitations

8.3.1 Evaluation on Synthetic Data Only

All quantitative results are based on synthetic data generated from the same pipeline used for training. This makes the evaluation controlled but optimistic. Real SOC data will be messier and more diverse. [10]

8.3.2 Limited Alert Type Coverage

Only five alert types were used in training. Commercial SIEMs cover hundreds. Expanding coverage is possible but requires additional data and retraining.

8.3.3 Simulated SIEM Connector

The system has a real Wazuh connector, but most evaluation runs use the simulator. A production integration would require careful mapping and testing against real SIEM payloads.

8.3.4 Manually Authored Policy Rules

Policy rules are configured manually. This requires domain expertise and tuning. There is no automated policy learning in the current implementation.

8.3.5 Feedback Loop Not Fully Closed

Feedback is captured in audit logs, but online weight updates are not integrated into the live inference loop. Retraining is still an offline step.

8.4 Threats to Validity

8.4.1 Internal Validity

Training and test data are generated by the same pipeline. This can inflate accuracy because the model learns generator patterns rather than real-world structure.

8.4.2 External Validity

Generalization to real SOC data is not proven. Distribution shift is a known challenge for security ML systems. [10]

8.4.3 Construct Validity

Strategy accuracy and action F1 measure label agreement, not operational impact. Whether the system reduces analyst workload or improves response time is not measured.

8.4.4 Statistical Validity

Results are reported from single training runs with fixed seeds. Multiple seeds and confidence intervals would strengthen the claims, but were not included in this work.

Conclusion and Future Work

9.1 Summary of Contributions

This thesis presents a generic SIEM reaction plugin that adds a reasoning layer to standard SIEM output. The system normalizes alerts, enriches context, scores risk with explanation, generates structured response plans, applies policy constraints, and records a full audit trail. A local intelligence service exposes multiple model backends through a single API, and a model registry enables rapid switching and comparison. The final planner is a lightweight custom neural network that achieves high accuracy while remaining fast enough for real-time use. The result is a practical, end-to-end pipeline that is explainable, auditable, and safe.

9.2 Future Research Directions

Several directions can strengthen the system and move it closer to production use. First, evaluation should be expanded to larger and more diverse real-world datasets, including mixed cloud and on-prem environments, to test generalization. Second, the intelligence layer should support drift detection and bounded continual learning so that model quality does not degrade silently over time.

Third, response planning can be improved by incorporating richer operational constraints such as change windows, business dependencies, and explicit approval workflows. Formal policy verification and stronger rollback semantics would further improve safety. Fourth, explainability should be aligned with standard security taxonomies so that explanations are consistent across systems and easier for analysts to trust.

Finally, the system should be validated through real SOC deployments. Longitudinal studies with analysts are needed to measure trust, workload reduction, and the real

operational impact of reasoning-driven response. These steps provide a clear path toward a production-grade reasoning layer for modern SIEM workflows.

References

- [1] Karen Kent and Murugiah Souppaya. Guide to computer security log management. Technical Report NIST SP 800-92, National Institute of Standards and Technology, 09 2006. URL <https://csrc.nist.gov/pubs/sp/800/92/final>.
- [2] Tao Ban, Takeshi Takahashi, Samuel Ndichu, and Daisuke Inoue. Breaking alert fatigue: Ai-assisted siem framework for effective incident response. *Applied Sciences*, 13(11):6610, 2023. doi: 10.3390/app13116610. URL <https://www.mdpi.com/2076-3417/13/11/6610>.
- [3] J. Tilbury and S. Flowerday. Humans and automation: Augmenting security operation centers. *Journal of Cybersecurity and Privacy*, 4(3):388–409, 2024. doi: 10.3390/jcp4030020. URL <https://www.mdpi.com/2624-800X/4/3/20>.
- [4] S. Sarhan, M. Haque, M. Asad, P. Kabo, A. Apon, and I. M. Elfadel. Rulegenie: Siem detection rule set optimization. *arXiv preprint arXiv:2505.06701*, 2025. doi: 10.48550/arXiv.2505.06701. URL <https://arxiv.org/abs/2505.06701>.
- [5] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. Computer security incident handling guide. Technical Report NIST SP 800-61 Rev. 2, National Institute of Standards and Technology, 2012. URL <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>.
- [6] Fredrik Valeur, Giovanni Vigna, Christopher Kruegel, and Richard A. Kemmerer. A comprehensive approach to intrusion detection alert correlation. *IEEE Transactions on Dependable and Secure Computing*, 1(3):146–169, 2004. doi: 10.1109/TDSC.2004.21.
- [7] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. MITRE ATT&CK: Design and philosophy. Technical Report MTR180002, The MITRE Corporation, 2018. URL https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf.

- [8] Stefan Axelsson. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security*, 3(3):186–205, 2000. doi: 10.1145/357830.357849.
- [9] Pedro Garcia-Teodoro, Jesus Diaz-Verdejo, Gabriel Macia-Fernandez, and Enrique Vázquez. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, 28(1–2):18–28, 2009. doi: 10.1016/j.cose.2008.08.003.
- [10] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pages 305–316. IEEE, 2010. doi: 10.1109/SP.2010.25.
- [11] Roberto Rodriguez and Jose Luis Rodriguez. OTRF security datasets: Structured attack telemetry for security research. GitHub repository, 2020. URL <https://github.com/OTRF/Security-Datasets>.
- [12] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018. doi: 10.5220/0006639801080116.
- [13] Defense Advanced Research Projects Agency. Transparent computing (TC) program. DARPA Information Innovation Office, 2019. URL <https://www.darpa.mil/research/programs/transparent-computing>.
- [14] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2305.14314>.
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [16] Paul Kassianik, Baturay Saglam, Alexander Chen, et al. Llama-3.1-FoundationAI-SecurityLLM-base-8b technical report. arXiv preprint arXiv:2504.21039, 2025. URL <https://arxiv.org/abs/2504.21039>.
- [17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu

- Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [18] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024. URL <https://arxiv.org/abs/2210.11416>.
- [19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1912.01703>.

Appendix

A.1 Additional Figures

A.2 Configuration Details